

SGLang-Omni · 每日 PR 学习日报

报告日期：2026-06-17 (北京时间) · 仓库：[sgl-project/sglang-omni](#) · 统计窗口：过去 24 小时内 merged PR

合并 PR 总数：6

深度讲解：3 (Omni / 多模态 / 多阶段)

简要覆盖：3 (CI / 工程基建)

主题：TTS 采样优化 · 多阶段显存预算 · 并发压测

6

24h 内 Merged PR

3

核心架构 / 性能

3

CI / 权限基建

2

涉及 TTS 模型 (Higgs / MOSS)

今日目录（讲解深度优先）

1. #816 [Higgs] 用 sgl_kernel 融合 top-k/top-p 采样 — 推理性能 / 多码本采样 / kernel 融合
2. #810 [MOSS] 调优 MOSS Local 同卡共置 AR 显存预算 — 多阶段流水线 / 显存管理
3. #770 [Qwen3-Omni] 并发 / rollout 压测脚本 — 多模态服务压测 / 可观测性
4. #814 / #806 / #727 — CI 与权限基建（合并讲解） — 工程基建

说明：本报告对前 3 个 PR 做“逐块代码 + 概念扫盲”式精讲，后 3 个 CI 类 PR 因技术含量较低，合并为一节简述。讲透 > 讲全。

#816

[Higgs] Fuse top-k/top-p sampling with sgl_kernel renorm

MERGED

多模态 TTS

性能优化

作者 @BBuf · 合并于 2026-06-17

08:22 UTC · [查看 PR](#)

1 PR 概览

- 标题：[Higgs] Fuse top-k/top-p sampling with sgl_kernel renorm
- 编号 / 作者 / 状态：#816 · @BBuf · 已合并 (**merged**)
- 链接：github.com/sgl-project/sglang-omni/pull/816
- 一句话：把 Higgs Audio v3 这个 TTS 模型在“逐步生成音频 token”时所用的 top-k/top-p 采样，从“全词表排序 + 掩码”的慢实现，换成 sgl_kernel 里写好的两个融合 CUDA kernel（`top_k_renorm_prob` / `top_p_renorm_prob`），在数值几乎完全等价的前提下把每步解码 GPU 时间降了约 1.7%。

2 它解决了什么问题

Higgs 是一个文本转语音（**TTS**）模型。它生成音频的方式不是直接吐波形，而是像 LLM 吐文字 token 一样，一步一步地吐“音频

token” (codebook token)。每一步都要从一个概率分布里“抽签”决定下一个 token，这个抽签过程就叫采样 (**sampling**)。

作者用性能分析工具 (profiler) 在 H100 上跑 Higgs 解码，发现一个痛点：每一个解码步，采样器都在做一条“全词表排序”的重活——

- 对整个词表 (vocab，这里 $V=1026$ ，而且有 8 个 codebook 同时采样) 做 `torch.sort` 完整排序；
- 再 `topk(K_MAX)` 取前 K 大；
- 再 `cumsum` (累加和) + `scatter` (散射写回) 来做 top-p 的掩码。

profiler 看到的具体 CUDA kernel 是 `radixSortKVInPlace`、`sbtok::gatherTopK`、`softmax_warp_forward` 一长串。问题在于：词表很小 (才 1026)，但这些 kernel 一个接一个地启动，每个都很短，GPU 大部分时间不是在算，而是在“等下一个 kernel 被启动”——这叫 **launch-bound** (启动开销受限)。就像让一个大厨切一片黄瓜，结果光是“拿刀、放刀、再拿刀”的动作比切本身还久。

类比 排序是为了找“前 K 大 / 累计概率到 p”，但其实你根本不需要把所有元素排好序，只需要知道“阈值在哪、谁该保留”。为此付出一次 $O(V \log V)$ 的全排序，是杀鸡用牛刀。融合 kernel 直接在一个 kernel 内部用更聪明的算法 (partial selection + 重归一化) 算出结果，省掉了排序和多次 kernel 往返。

3 具体做了什么改动

唯一改动的文件：`sglang_omni/models/higgs_tts/sampler.py`，函数 `_sample_independent_batched`。下面把改动前和改动后对照讲清楚。

▼ 改动前 (旧逻辑：排序 + 掩码)

```
# top-k: 对 logits 做 topk, 取第 k 大的值作为阈值 kth,
# 小于阈值的位置全部置为 -inf (即丢弃)
if top_k_buf is not None:
    top_vals = logits.topk(K_MAX, dim=-1).values
    k_idx = top_k_buf.view(B,1,1).expand(-1,N,1).clamp(min=1,max=logits.shape[-1])
    kth = top_vals.gather(-1, k_idx)
    logits = torch.where(logits < kth, float("-inf"), logits)

# top-p (核采样) : 先全量排序, 算累计概率 cumsum,
# 累计超过 p 的尾部全部丢弃, 再 scatter 写回原位置
if top_p is not None:
```

```

sorted_logits, sorted_indices = torch.sort(logits, descending=True)
cum_probs = sorted_logits.softmax(dim=-1).cumsum(dim=-1)
remove = cum_probs > top_p.view(B,1,1)
remove[..., 1:] = remove[..., :-1].clone() # 右移一位, 保留原
remove[..., 0] = False
scatter = torch.zeros_like(remove)
scatter.scatter_(-1, sorted_indices, remove) # 把“该
logits = torch.where(scatter, float("-inf"), logits)

probs = logits.softmax(dim=-1)
codes_flat = probs.reshape(B*N, V).multinomial(num_samples=1).

```

▼ 改动后（新逻辑：先 softmax 成概率，再调融合 kernel 重归一化）

```

# 关键：先把 logits 变成概率分布, 强制 contiguous + fp32 (kernel 硬件
probs = logits.float().softmax(dim=-1).reshape(B*N, V).contiguous()

if top_k_buf is not None:
    tk = top_k_buf.view(B,1).expand(B,N).reshape(B*N) \
        .clamp(min=1,max=V).to(torch.int32).contiguous()
    probs = _fused_top_k_renorm(probs, tk) # 一个 kernel

if top_p is not None:
    tp = top_p.view(B,1).expand(B,N).reshape(B*N).to(torch.float)
    probs = _fused_top_p_renorm(probs, tp) # 一个 kernel

codes_flat = probs.multinomial(num_samples=1).squeeze(-1) # 抽

```

逐块解读两者的差别：

- 旧逻辑在 **logits** 空间操作：用排序找阈值、用 `-inf` 掩码、最后才 softmax。中间产生 `sorted_logits`、`sorted_indices`、`scatter` 等一堆临时大张量，且每个 PyTorch 算子都是一次独立 kernel 启动。
- 新逻辑在 **probability** 空间操作：先 softmax 成概率，然后直接交给 `_fused_top_k_renorm` / `_fused_top_p_renorm`。这两个是 sgl_kernel（背后是 FlashInfer）提供的 CUDA kernel，它们内部一次性完成“筛选 + 把剩下的概率重新归一化到和为 1”，不再需要 PyTorch 层面的排序和散射。
- 为什么叫 **renorm**（重归一化）：当你把一部分 token 的概率砍掉后，剩下概率之和不再是 1，必须重新除以新的总和，`multinomial` 抽签才正确。融合 kernel 把“砍 + 归一化”合并成了一步。

- 三个工程细节（很关键，新手最容易踩坑）：
 - `.contiguous()` 和 `.float()`：FlashInfer 的 renorm kernel 硬性要求输入是连续内存的 **fp32** 张量。PR 描述里特别强调“非连续张量会静默地破坏结果”——即不报错但算错，这是最阴险的 bug，所以必须显式 `contiguous`。
 - `.to(torch.int32)`：top_k 参数要转成 int32，因为 kernel 接口签名是这么定义的。
 - **greedy / argmax** 短路和 **CUDA graph** 安全的 **multinomial** 保持原样没动——作者只换了中间这段，最大程度减少风险面。

4

涉及的知识点（扫盲精讲）

框架 / 领域

TTS 的自回归 token 生成 & codebook（码本）

是什么：现代神经 TTS（如 Higgs、MOSS）不直接生成波形，而是先用一个“音频编解码器（neural audio codec，如 EnCodec/SNAC 类）”把声音离散化成一串整数 token（就像把图片压成 JPEG 的系数）。模型自回归地预测这些 token，最后再用解码器（vocoder）还原成 24kHz 波形。**codebook** 就是这些离散 token 的“字典”；Higgs 用 8 个 codebook 并行（multi-codebook），所以一步要同时为 8 路各采一个 token。

为什么需要：把连续音频离散化后，就能完全复用 LLM 那一套“预测下一个 token”的成熟基础设施（KV cache、batching、采样）。

本 PR 里怎么用：被优化的 `_sample_independent_batched` 就是“为这 8 路 codebook 各自独立采样”的函数，形状 `B×N×V` 里 N 就是 codebook 数。

采样算法

top-k / top-p（核采样）与温度（temperature）

是什么：模型每步输出一个对全词表的打分（logits）。直接按概率抽签容易抽到长尾垃圾 token；于是用三道“质量闸门”：**temperature** 调节分布陡峭程度（越低越保守）；**top-k** 只保留概率最高的 k 个候选；**top-p（nucleus）** 保留“累计概率达到 p”的最小候选集。

为什么需要：在 TTS 里直接关系到音质——采样太“野”会爆音、跑调；太“死”会机械、单调。

本 **PR** 里怎么用：算法语义完全不变，只是把实现从“排序版”换成“融合 kernel 版”。作者特别说明：唯一理论差异在 `cumsum == top_p` 的精确边界上，旧代码用 `cum > p` 再右移，新 kernel 用标准 `cum >= p` 约定；对连续 logits 这是“测度为零”事件（几乎不可能精确相等），实测 225 种配置里从未触发。

性能工程

Kernel 融合 (fusion) 与 launch-bound

是什么：GPU 每执行一个算子就要“启动一个 kernel”，启动本身有固定开销（几微秒）。当单个算子的计算量很小，启动开销就会主导总时间——这就是 **launch-bound**。**Kernel** 融合是把多个小算子合并成一个大 kernel，用一次启动完成多步计算。

为什么需要：Higgs 的词表只有 1026，每个排序/掩码算子算得飞快但启动一堆，典型的“启动比计算贵”。

本 **PR** 里怎么用：把 `sort+topk+cumsum+scatter+softmax` 这一长串，压成 `softmax + 1~2 个 renorm kernel`。profiler 显示采样 kernel 块从 $\sim 64\mu\text{s}/\text{步}$ 降到 $\sim 32\mu\text{s}/\text{步}$ （砍半）。

类比 就像点外卖：与其分别下 5 个单（5 次配送费），不如合并成 1 单。计算（菜）没变，省的是“配送费”（kernel 启动）。

推理基建

sgl_kernel / FlashInfer 与 CUDA Graph

是什么：`sgl_kernel` 是 SGLang 自带的高性能算子库，底层很多来自 **FlashInfer**（专为 LLM 推理写的 CUDA kernel 集合）。**CUDA Graph** 则是把一串固定的 GPU 操作“录制”成一张图，之后整图一次性重放，进一步消除 CPU 端逐 kernel 启动的开销。

为什么需要：解码阶段每步操作高度重复，CUDA Graph 能把上百次 launch 压成一次重放，是 LLM/TTS 解码提速的标配。

本 **PR** 里怎么用：作者验证了替换后“CUDA graph 捕获不受影响、端到端仍输出有效 24kHz 语音、无 graph break”。所谓 **graph break** 指

引入了不能被图捕获的动态操作（如数据相关的控制流），会迫使图断开、退回慢路径——所以保持 multinomial 那段“graph-safe”很重要。

5 性能数据 & 我能学到什么

指标（H100, n=2, 固定请求）	baseline	本 PR	变化
每步解码 GPU 时间 (μs/step)	4152.1	4082.8	-1.67%
采样 kernel 块 (μs/step)	~64	~32	≈ 砍半
最大概率差 (vs 旧实现)	4.8e-7 (fp32 精度级)		数值等价

可复用的工程思想：

- 先 **profile** 再优化：作者不是凭感觉，而是用 profiler 定位到“采样器在 launch-bound”，再针对性下手。优化的第一步永远是测量。
- 等价替换 + 严格验证：用 225 种 (B × temperature × top_k × top_p) 组合做扫描，证明 max prob diff ~5e-7、保留集合完全一致，把“性能优化引入正确性回归”的风险压到最低。这是工业级优化 PR 的范式。
- 最小改动面：只动中间一段，greedy 短路 / multinomial / CUDA graph 一律不碰。
- 魔鬼在细节：contiguous + fp32 这种“不满足会静默算错”的隐性契约，必须在代码和注释里显式处理。

想深入该补的前置知识：GPU 执行模型（SM / warp / kernel launch 开销）、softmax 与采样数值稳定性、FlashInfer 的 top-k/top-p renorm 实现、CUDA Graph 的捕获条件与 graph break 触发场景。

#810

[MOSS] Tune MOSS Local colocated AR memory budgeting

1 PR 概览

- 编号 / 作者 / 状态 : #810 · @Ratish1 · 已合并 (**merged**)
- 链接 : github.com/sgl-project/sglang-omni/pull/810
- 一句话 : MOSS Local 这个 TTS 流水线在单卡上把“预处理 + 自回归(AR) + 声码器(vocoder)”三个阶段挤在同一张 GPU 上跑 (colocated, 共置)。本 PR 给中间最吃显存的 AR 阶段一套更精确、显式的显存预算方案——用“进程级显存核算”代替 SGLang 默认的“整卡空闲显存”估算, 并预留一小块给后面加载的 codec, 端到端吞吐 +8.9%、WER 还略降。

2 它解决了什么问题

先理解多阶段流水线 (**multi-stage pipeline**) : MOSS Local 把一次 TTS 请求拆成三段串行处理——

- **preprocessing** (预处理) : 处理文本、参考音色等, 会加载一个 codec (编解码器)。
- **tts_engine / AR** (自回归) : 核心大模型, 逐 token 生成音频 token, 最吃显存 (要放权重 + KV cache)。
- **vocoder** (声码器) : 把 token 还原成波形, 又要加载一个 codec 实例。

在单卡部署里, 这三段共用同一张 **GPU (colocated)**。痛点来了 : SGLang 引擎启动时要决定“我能用多少显存放 KV cache”, 它默认的做法是看整张卡当前还剩多少空闲显存, 按比例 `mem_fraction_static` 切一块。但在共置场景下, 这个“整卡空闲”是不准的——预处理的 codec 已经占了一块、后面 vocoder 的 codec 还没加载但迟早要占。如果 AR 阶段贪心地按“当前空闲”吃满, 等 vocoder 来加载 codec 时就 **OOM** (显存爆掉)。

旧代码的做法是“拍脑袋”给个静态数 : `mem_fraction_static = 0.6 (多卡) 或 0.5 (单卡)`——保守、浪费, 而且没有理论依据。本 PR 要做的是 : 把这套预算显式化、结构化、并接入更准的“进程级显存核算”。

类比 三个室友合租一间房（一张 GPU）。老办法是老二（AR）进门看“现在地上还空多少”就铺多大的床，结果老三（vocoder）回来发现没地方睡了（OOM）。新办法是事先按合同分配：总共用 90% 的房间，给老三的家具预留 5%，老二自己实际用 85%——并且用“房产证上的精确面积（进程级核算）”而不是“目测空地”。

3 具体做了什么改动

改动两个核心文件 + 一个测试文件：

(A) `config.py` — 把预算参数显式化、区分共置/分卡两种拓扑

```
# 新增三个显式常量，取代散落的魔数
_COLOCATED_TOTAL_GPU_MEMORY_FRACTION = 0.90 # 该阶段总共可用整卡
_COLOCATED_CODEC_MEM_RESERVE          = 0.05 # 给后续 vocoder 预留
_AR_MEM_FRACTION_STATIC                 = 0.85 # 分卡场景下 AR 的静态预算

def _stages(*, codec_device: str, colocated: bool):
    tts_engine_runtime = StageRuntimeConfig(
        resources=StageResourceConfig(
            # 共置：传入“总预算”，让引擎用进程级核算自己 profile
            total_gpu_memory_fraction=(_COLOCATED_TOTAL_GPU_MEMORY_FRACTION if colocated else 1),
        ),
        sglang_server_args=SGLangServerArgsConfig(
            # 分卡：沿用旧的静态 0.85；共置：交给 total_gpu_memory_fraction
            mem_fraction_static=(None if colocated else _AR_MEM_FRACTION_STATIC),
        ),
    )
    tts_engine_args = {"gpu_id": 0, "dtype": "bfloat16"}
    if colocated:
        tts_engine_args["codec_mem_reserve"] = _COLOCATED_CODEC_MEM_RESERVE
    ...
```

解读：默认单卡管线（`MossTTSLocalPipelineConfig` / ...Colocated...）走 `colocated=True` 这条新路；两卡变体（...Split...，codec 放到第二张卡 `cuda:1`）走 `colocated=False` 保持旧的静态预算。一份 `_stages()` 工厂函数，靠一个布尔开关产出两种拓扑配置——避免代码重复。

(B) `stages.py` — 新增预算计算函数

`_apply_colocated_ar_memory_budget`

```

def _apply_colocated_ar_memory_budget(overrides, *, total_gpu_memory_fraction=None):
    if total_gpu_memory_fraction is None:
        # 分卡/旧路：不
        return _ArMemoryBudget(None, 0.0)
    if not 0.0 <= codec_mem_reserve < 1.0:
        raise ValueError("codec_mem_reserve must be in [0, 1)")

    # 有效预算 = 总预算 - 给 codec 的预留。0.90 - 0.05 = 0.85
    effective = round(total_gpu_memory_fraction - codec_mem_reserve, 2)
    if effective < 0.1:
        # 安全下限，预留太多会
        raise ValueError("... below the safe floor 0.1 ...")

    # 若用户显式给了 mem_fraction_static，尊重它，但不许超过总预算
    explicit = overrides.get("mem_fraction_static")
    if explicit is not None:
        if explicit > total_gpu_memory_fraction:
            raise ValueError("... cannot exceed total_gpu_memory_fraction ...")
        effective = explicit
        applied_reserve = round(total_gpu_memory_fraction - effective, 2)
    else:
        overrides["mem_fraction_static"] = effective
    return _ArMemoryBudget(effective, applied_reserve)

```

(C) 工厂函数里接入“进程级核算”并提供安全回退

```

profile_total = memory_budget.effective_total_gpu_memory_fraction
if profile_total is not None:
    from sglang_omni.utils.gpu_memory import get_process_gpu_memory_bytes
    # 如果这台机器拿不到“本进程占了多少显存”这个数，就别用进程级预算，
    # 退回 SGLang 上游默认的“整卡空闲”profiling，避免用了不安全的预算
    if get_process_gpu_memory_bytes(gpu_id) is None:
        logger.warning("process memory accounting unavailable; falling back to default")
        profile_total = None
    ...
create_sglang_infrastructure(..., total_gpu_memory_fraction=profile_total)

```

解读三层逻辑：① 算出有效预算 0.85；② 把它透传给

`create_sglang_infrastructure`，让 SGLang 在“进程级显存”可用时按这个比例 `profile`；③ 如果环境拿不到进程级数据（比如某些容器/驱动），优雅降级回上游默认估算，绝不在不确定时硬上一个危险预算。这套“尝试更优 → 不行就安全回退”的写法非常值得学。

系统架构

多阶段流水线 **colocated**（共置） vs **split**（分卡）

是什么：一个 Omni/TTS 系统由多个模型阶段组成。共置=全部塞进一张 GPU（省卡、省成本，但显存紧张、互相争用）；分卡=把不同阶段放到不同 GPU（宽裕但费卡）。

为什么需要：生产部署要在“成本”和“稳定性”间权衡。单卡共置最便宜，但显存预算稍有不慎就 OOM——正是本 PR 要根治的。

本 PR 里怎么用：用一个 **colocated** 布尔，让同一份 stage 工厂同时支持两种拓扑，并只给共置路打上更精细的预算。

推理基建（核心）

KV cache 与 **mem_fraction_static** / 显存 **profiling**

是什么：自回归生成时，每个已生成 token 的 Key/Value 向量要缓存起来供后续步骤注意力复用，这就是 **KV cache**，它随序列变长而增大，是推理显存的大头。SGLang 启动时会做一次“显存 profiling”：先加载模型权重，剩下的显存按 **mem_fraction_static** 预先静态分配成一个大 KV cache 池（避免运行时反复申请/碎片化）。

为什么需要：预分配一个固定池能让显存可控、避免运行中 OOM，也利于做 continuous batching。但“按整卡空闲算”在共置场景会高估可用量。

本 PR 里怎么用：把“整卡空闲”估算换成“进程级核算（只算本进程实际占用）+ 显式预留 codec”，让 KV cache 池的大小既不浪费也不踩 OOM。

显存核算

进程级显存核算（**process-scoped accounting**） vs 整卡空闲

是什么：“整卡空闲”= 看 **nvidia-smi** 那种全卡剩余；“进程级”= 精确知道当前这个 **Python** 进程自己占了多少。共置时同卡上别的阶段也在占显存，只有进程级数才能让 AR 准确知道“我还能安全吃多少”。

本 PR 里怎么用：通过 `get_process_gpu_memory_bytes(gpu_id)` 判断该机制是否可用；可用就走精确预算，不可用就 warning + 回退。

设计模式

不可变值对象 + 显式校验 + 优雅降级

是什么：用 `@dataclass(frozen=True)` 的 `_ArMemoryBudget` 把“算出来的预算结果”打包成一个只读对象传递；所有边界（`reserve ∈ [0,1)`、`effective ≥ 0.1`、`explicit ≤ total`）都显式 `raise ValueError` 校验；环境不支持时不崩溃而是降级。

为什么需要：显存预算错了会直接 OOM 崩服务，属于“失败代价很高”的配置，必须把非法值在启动期、配置期就拦下来，而不是运行到一半才炸。

5 性能数据 & 我能学到什么

指标 (SeedTTS 英文, 1088 样本, 并发 8)	upstream/main	本 PR	变化
吞吐 Throughput (req/s)	4.772	5.199	+8.9%
RTF 均值 (越低越好)	0.4014	0.3677	-8.4%
延迟 p99 (s)	3.131	2.660	-15.0%
输出吞吐 (tok/s)	262.5	286.0	+9.0%
WER 语料级 (越低越好)	2.29%	2.08%	-0.21pp

RTF (Real-Time Factor, 实时率) = 合成耗时 / 音频时长, <1 表示比实时还快；

WER (Word Error Rate, 词错误率) = 把合成语音再转写回文字与原文的差异，衡量音质/可懂度。两者同时变好说明：把显存预算调对后，能塞下更大的 KV cache → 支持更高并发 → 吞吐涨，而音质不退反进。

可复用的工程思想：

- “魔数”要变“具名常量 + 显式预算公式”：旧的 0.5/0.6 拍脑袋值升级成 `total - reserve = effective` 的可解释公式。

- 配置层做强校验，失败要趁早：把会导致 OOM 的非法预算在配置期 `raise`，而不是运行期崩。
- “能力探测 + 优雅降级”：先探测进程级核算是否可用，不可用就回退到安全默认，保证在各种环境都不崩。
- 一份工厂、布尔开关产出多拓扑：colocated/split 复用同一段 `_stages`，减少分叉。
- 改配置也要写单测：PR 为“默认/分卡 wiring、预算透传、无进程核算回退、显式预算校验”都加了单测。

想深入该补的前置知识：GPU 显存分配器与碎片、KV cache 容量如何决定最大并发与 batch、continuous batching 原理、bf16 权重的显存占用估算、多进程共享单卡的显存隔离。

#770

[Qwen3-Omni] benchmarks: concurrency / rollout stress runner

MERGED

多模态 Qwen3-Omni

压测·可观测性

作者 @luojiaxuan ·

合并于 2026-06-16 04:45 UTC · [查看 PR](#)

1 PR 概览

- 编号 / 作者 / 状态：#770 · @luojiaxuan · 已合并 (merged)
- 链接：github.com/sgl-project/sglang-omni/pull/770
- 一句话：新增一个独立的并发/**rollout** 压测脚本
(`benchmark_omni_rollout_stress.py`)，用同一个多模态提示词在 1→2→4→8→16→32 不同并发级别下反复轰炸 Qwen3-Omni 服务，记录每档的 req/s、输出 tok/s、p50/p95/p99 延迟，外加服务端 profiler 事件——纯诊断工具，不改任何运行时代码。

2 它解决了什么问题

Qwen3-Omni 是一个全模态 (**Omni**) 大模型：能吃文本+图像+音频，还能吐文本+语音。要把它放到生产里服务，工程师必须回答一个问题：“当很多请

求同时打进来，它会怎么表现？吞吐能不能线性涨？延迟会不会爆？”这正是 issue #760（并发服务表现）要刻画的。

痛点：在此之前缺一个可复现的、标准化的压测手段来量化“并发扩展性”。尤其是 **rollout** 场景——同一个 prompt 被复制 N 份并发跑（强化学习采样、多候选生成里极常见），这种“同提示多路并发”有它独特的缓存/调度行为，需要专门工具刻画 before/after 数字。

作者特意强调它放在 `benchmarks/stress_tests/`（压测/诊断）而非 `benchmarks/eval/`（能力评测），避免被误当成标准精度指标——这是很清醒的工程边界意识。（注：合并入库时落在了 `benchmarks/eval/benchmark_omni_rollout_stress.py`。）

3 具体做了什么改动

纯新增一个文件，约 333 行。核心是一个 `asyncio` 异步压测器。逐块看关键逻辑：

(A) 把同一个样本复制 **N** 份，制造“同提示多路并发”

```
def _duplicate_prompt(sample: MMMUSample, count: int):
    return [
        replace(sample, sample_id=f"{sample.sample_id}:rollout-n{count}for index in range(count)
    ]
```

解读：`dataclasses.replace` 复制一份样本只改 `sample_id`（保证每路有唯一 ID 便于追踪），prompt 内容完全相同——这就模拟出 rollout 那种“同一输入并发多路”。

(B) 单个请求如何发：构造多模态 **payload** + 解码音频校验

```
payload = {
    "model": model_name,
    "messages": [{"role": "user", "content": sample.prompt}],
    "images": [image_to_data_uri(img) for img in sample.images], #
    "modalities": ["text", "audio"], # 要求同时输出文本和语音
    "max_tokens": max_tokens, "stream": False,
    "user": rollout_group_id, # 同组打同一个 user 标记
}
if enable_audio:
```

```

payload["audio"] = {"format": "wav"}
...
# 拿到响应后, 把 base64 音频解码一次——只为暴露传输/编码损坏,
# 又不必为每路 rollout 真写一个 WAV 文件 (省 IO)
base64.b64decode(audio_b64)

```

解读：这段体现了对全模态输出的正确处理——请求里

`modalities=["text","audio"]` 表示要模型既回文本又回语音；返回的语音是 base64 编码的 WAV，作者只 `b64decode` 一次做“健康检查”（坏数据会在这里抛错），但不落盘，把压测的 IO 噪声降到最低。这是压测脚本的讲究：测的是服务，不要让客户端自己成为瓶颈。

(C) 主循环：逐档并发级别压测 + 收集指标

```

for count in args.rollout_counts:          # 1,2,4,8,16,32 逐档加压
    duplicated = _duplicate_prompt(base_sample, count)
    runner = BenchmarkRunner(RunConfig(
        max_concurrency=count,
        request_rate=float("inf"),          # 闭环：不限速，一有空位立刻补请
        warmup=0,
    ))
    request_results = await runner.run(duplicated, send_fn)
    speed = compute_speed_metrics(request_results, wall_clock_s=run_time)
    print_speed_summary(speed, args.model, count, ...) # 打印 req/

```

解读：`request_rate=inf` + `max_concurrency=count` 构成一个闭环

(closed-loop) 压测——不是按固定速率发请求（开环），而是“维持 count 个并发在飞，一个完成立刻补一个”，这样测出的是服务在给定并发压力下的饱和吞吐。

(D) 包裹服务端 profiler：定位瓶颈在哪一段

```

if not args.no_profile:
    await _start_request_profile(session, base_url, run_id=run_id,
    try:
        ... # 跑所有并发档
    finally:
        if not args.no_profile:
            await _stop_request_profile(session, base_url, run_id=run_id)

```

解读：通过 HTTP 调服务端的 `/start_request_profile` 和 `/stop_request_profile`，让服务端在压测期间录制内部事件（哪段排队、哪段算）。注意用 `try/finally` 保证即使压测中途异常，profiler 也一定被关掉——否则会一直录、泄漏资源。压测前后还各打一次 `/health`，对比服务有没有被压挂。

4 涉及的知识点（扫盲精讲）

领域概念

Qwen3-Omni 与“全模态 (any-to-any)”服务

是什么：Omni 模型能接收多种模态（文+图+音）并生成多种模态（文+语音）。在 SGLang-Omni 里，它常是一个多阶段系统：thinker（理解+生成文本）→ talker（把文本转成语音 token）→ codec/vocoder（出波形）。

本 PR 里怎么用：payload 里 `modalities=["text","audio"]`、带 `images`、带 `talker_max_new_tokens`，正是在压测这条“图文进、文+语音出”的全链路。

服务压测

闭环(closed-loop) vs 开环(open-loop)、并发 vs 速率

是什么：开环按固定 QPS 发请求（模拟真实用户到达，关心“能否扛住 X QPS”）；闭环维持固定在飞并发数（关心“满负荷下的极限吞吐/扩展性”）。

为什么需要：要刻画“并发扩展性曲线”，闭环最直接——逐档抬高并发，看吞吐是否线性增长、延迟何时拐点爆炸。

本 PR 里怎么用：`request_rate=inf` + `max_concurrency=count` = 闭环；扫 1/2/4/8/16/32 得到扩展性曲线。

ROLLOUT

同提示多路 rollout 与 KV / 前缀缓存

是什么：rollout 指同一输入并发跑多条生成（RL 训练采样、best-of-n 都靠它）。同提示意味着共享前缀，理论上可被 SGLang 的 RadixAttention / 前缀缓存复用，省掉重复的 prefill 计算。

为什么需要压它：同提示多路是否真能高效复用前缀、调度会不会打架，直接决定 RL 数据生成的成本。

本 **PR** 里怎么用： `_duplicate_prompt` 造出 N 份同提示并打同一 `user/rollout_group_id`，专门压这种流量。

指标

p50/p95/p99 尾延迟、req/s、tok/s 与 profiler

是什么：均值会骗人，**p95/p99** 尾延迟才反映“最差那批用户的体验”；`req/s` 是吞吐，`tok/s` 是生成速度。**profiler** 事件则把端到端时间拆成“排队/prefill/decode”各段，定位瓶颈。

本 **PR** 里怎么用：每档都 `compute_speed_metrics` + `print_speed_summary` 输出这些数，结果连同 health/profile 存成 `rollout_stress_results.json` 供前后对比。

异步编程

asyncio + aiohttp 做高并发客户端

是什么：压测客户端要同时维持几十路在飞请求，用线程会很重；`asyncio` 协程 + `aiohttp` 异步 HTTP 能用单线程高效管理大量并发连接（IO 等待时切去做别的）。

本 **PR** 里怎么用：`send_fn` 是 `async` 协程，`BenchmarkRunner` 用 `max_concurrency` 控制同时在飞的协程数；超时用 `aiohttp.ClientTimeout` 兜底。

可复用的工程思想：

- **诊断工具与评测工具要分家**：放在 `stress_tests/` 而非 `eval/`，避免把压测数字误读成模型精度——清晰的目录语义就是清晰的工程契约。
- **不改运行时、只加观测**：本 PR “零运行时代码改动”，把测量和被测系统解耦，风险极低却能产出决策依据。
- **客户端别成为瓶颈**：音频只 decode 不落盘、用 `asyncio` 而非线程，确保测的是服务端而非客户端。
- **资源用 `try/finally` 收尾**：profiler 一定关、health 前后各探一次——压测脚本的健壮性细节。
- **大 PR 拆小**：这是 #769 拆成的三连之一（#770 工具 / #771 prefill 批处理+TP 修复 / #772 准入+遥测），单一职责、易 review。

想深入该补的前置知识：RadixAttention / 前缀缓存如何复用同提示、continuous batching 调度、开环 vs 闭环压测方法论、Little's Law（并发=吞吐×延迟）、`asyncio` 事件循环、Qwen3-Omni 的 thinker-talker 多阶段结构。

CI

#814 / #806 / #727 — CI 与权限基建（合并简述）

MERGED ×3

工程基建

技术含量较低，合并为一节

这三个 PR 都属于 **CI** / 权限基建，不触及模型或推理逻辑，故合并简述：

- **#727 · feat(ci): add MOSS TTS to TTS CI with random model selection** (@yijiangt, 关联 issue #724)：把 TTS CI 从“只测 Higgs”扩展成多模型随机抽一个测（Higgs 或 MOSS），保持每次提交的 CI 成本不变。新增 `pick-tts-model` 任务随机选模型，引入

`TtsCiModelPreset` 预设；MOSS 走 `gate_thresholds=False`（只测量打印、不卡 CI）。学习点：在固定算力预算下用“随机抽样”扩大覆盖面，是 CI 设计的实用技巧；新增模型只需改 3 处，扩展性好。

- **#814 · [CI] Enable Labels for TTS CI** (@zhaochenyang20)：让 TTS CI 支持 label 触发（打 `run-ci` 等标签才在自建 GPU runner 上跑），节省宝贵的 GPU CI 资源。
- **#806 · [CI] Add luojiaxuan to CI permissions** (@luojiaxuan)：在 `.github/Ci_PERMISSIONS.json` 给某贡献者授予 `/tag-run-ci-label` 和 `/rerun-failed-ci` 权限，并用脚本校验 JSON 合法性。

这三者共同反映 SGLang-Omni 的一个工程现实：**GPU CI** 很贵，所以围绕“标签触发、随机抽测、细粒度权限”做了一整套省钱又可控的 CI 治理。

今日整体洞察（把 6 个 **PR** 串起来看）：这一天的合并清晰地勾勒出 SGLang-Omni 的两条主线——① **推理性能与显存的精打细算**（#816 用 kernel 融合省采样开销、#810 用进程级核算把单卡共置的显存预算调对）；② **可观测性与 CI 治理**（#770 给 Qwen3-Omni 补并发压测工具、#727/#814/#806 把昂贵的 GPU CI 管得更省更细）。两条线背后是同一个工程哲学：先测量、再优化；改动要等价且最小、配置要强校验、能力探测后优雅降级。对一个服务多种 TTS/Omni 模型的多阶段推理框架来说，这正是把“能跑”推向“跑得稳、跑得省”的日常功夫。

本报告由自动化日报例程于 2026-06-17 生成 · 数据源：GitHub `sgl-project/sglang-omni` · 仅供学习参考

讲解深度优先于覆盖广度：3 个核心 PR 精讲 + 3 个 CI PR 简述。